Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

LAB 10 : Mini Project

Name: Ananya Godse **SAP ID:** 60009220161 **Batch:** D1-1

Project Title: Real-Time Vehicle Detection and Directional Tracking on Highway Traffic Using YOLOv8

Introduction:

This project involves real-time vehicle detection and tracking on a highway using a pre-trained YOLO (You Only Look Once) object detection model. The system identifies and counts vehicles such as cars, buses, and trucks moving up and down in two lanes, providing valuable data for traffic monitoring and analysis.

Objectives:

- To detect different types of vehicles (cars, buses, trucks) in a video stream.
- To track the direction of each vehicle (up or down) using reference lines.
- To count the vehicles moving in each direction, distinguishing between types.
- To display the FPS (Frames Per Second) to assess real-time processing performance.

Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

Code:

main.py

main.py tracker.py

main.py > ...

```python
import cv2
import pandas as pd
from ultralytics import YOLO
import cvzone
import time
from tracker import*


# load the pre-trained yolov8 model
model = YOLO('yolov8s.pt')

# define mouse callback function to track mouse position over an OpenCV window
def track_mouse_position(event, x, y, flags, param):
    if event == cv2.EVENT_MOUSEMOVE:
        point = [x, y]
        print(point)

# create an OpenCV window named 'window' to display video frames
cv2.namedWindow('window')
# assign track_mouse_position function to 'window'
cv2.setMouseCallback('window', track_mouse_position)

# open the video file
capture = cv2.VideoCapture('vehicles_video.mp4')

# get classes from coco.txt
coco_file = open("coco.txt", "r").read()
class_list = coco_file.split('\n')

frame_count = 0 # keep count of frames

```

Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

```python
# create instances of Tracker class for car, bus & truck
car_tracker = Tracker()
bus_tracker = Tracker()
truck_tracker = Tracker()

# represent two horizontal lines in the frame - used as thresholds to determine the direction of vehicle movement.
cy1 = 184
cy2 = 209

# provides a buffer area around the threshold lines.
offset = 8

# defining dictionaries and lists to count and track specific vehicles going up or down
upcar = {}
downcar = {}
upcar_counter = []
downcar_counter= []

upbus = {}
downbus = {}
upbus_counter = []
downbus_counter = []

uptruck = {}
downtruck = {}
uptruck_counter = []
downtruck_counter = []

# start time for FPS calculation
start_time = time.time()
```

```python
# process video frames in a loop, resizing and analyzing every third frame with YOLO object detection
while True:
    ret, frame = capture.read()
    if not ret:
        break

    frame_count += 1

    if frame_count % 3 != 0:
        continue

    frame = cv2.resize(frame, (1020, 500))

    # use the YOLO model to detect objects
    results = model.predict(frame)

    # extract the bounding box and store it in 'px' dataframe
    a = results[0].boxes.data
    px = pd.DataFrame(a).astype("float")

    # initialize lists to hold bounding boxes for cars, buses & trucks.
    car_list = []
    bus_list = []
    truck_list=[]

    # iterate over detected bounding boxes
    for index, row in px.iterrows():
        x1 = int(row[0])
        y1 = int(row[1])
        x2 = int(row[2])
        y2 = int(row[3])
```

Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

```
        detected_index = int(row[5])
        classes = class_list[detected_index]
        if 'car' in classes:
            car_list.append([x1, y1, x2, y2])

        elif 'bus' in classes:
            bus_list.append([x1, y1, x2, y2])

        elif 'truck' in classes:
            truck_list.append([x1, y1, x2, y2])


    # draws a bounding box with a circle at the center and labels the object ID on the given frame.
    def draw_bounding_box(frame, bbox, color, text_prefix):
        x3, y3, x4, y4, id = bbox
        cx3 = int(x3 + x4) // 2
        cy3 = int(y3 + y4) // 2

        cv2.circle(frame, (cx3, cy3), 4, color[0], -1)
        cv2.rectangle(frame, (x3, y3), (x4, y4), color[1], 2)
        cvzone.putTextRect(frame, f'{text_prefix} ID: {id}', (x3, y3), 1, 1)


     # update the car tracker and draw bounding boxes
    bbox_idx_car = car_tracker.update(car_list)
    for bbox in bbox_idx_car:
        draw_bounding_box(frame, bbox, ((255, 0, 0), (255, 0, 255)), 'car')
        center_y = (bbox[1] + bbox[3]) // 2
```

```
            # track vehicle direction
            if center_y < cy1 - offset and bbox[4] not in upcar:
                # if vehicle is going up and hasn't been counted up yet
                if bbox[4] not in downcar:
                    upcar[bbox[4]] = True
                    upcar_counter.append(bbox[4])
            elif center_y > cy2 + offset and bbox[4] not in downcar:
                # if vehicle is going down and hasn't been counted down yet
                if bbox[4] not in upcar:
                    downcar[bbox[4]] = True
                    downcar_counter.append(bbox[4])

        bbox_idx_bus = bus_tracker.update(bus_list)
        for bbox in bbox_idx_bus:
            draw_bounding_box(frame, bbox, ((255, 255, 0), (255, 255, 0)), 'bus')
            center_y = (bbox[1] + bbox[3]) // 2

            if center_y < cy1 - offset and bbox[4] not in upbus:
                if bbox[4] not in downbus:
                    upbus[bbox[4]] = True
                    upbus_counter.append(bbox[4])
            elif center_y > cy2 + offset and bbox[4] not in downbus:
                if bbox[4] not in upbus:
                    downbus[bbox[4]] = True
                    downbus_counter.append(bbox[4])

        bbox_idx_truck = truck_tracker.update(truck_list)
        for bbox in bbox_idx_truck:
            draw_bounding_box(frame, bbox, ((0, 255, 255), (0, 255, 255)), 'truck')
            center_y = (bbox[1] + bbox[3]) // 2
```

Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

```
        if center_y < cy1 - offset and bbox[4] not in uptruck:
            if bbox[4] not in downtruck:
                uptruck[bbox[4]] = True
                uptruck_counter.append(bbox[4])
        elif center_y > cy2 + offset and bbox[4] not in downtruck:
            if bbox[4] not in uptruck:
                downtruck[bbox[4]] = True
                downtruck_counter.append(bbox[4])


    # draw reference lines on the frame for traffic monitoring
    cv2.line(frame, (1, cy1), (1018, cy1), (0, 255, 0), 2)  # green reference line
    cv2.line(frame, (3, cy2), (1016, cy2), (0, 0, 255), 2)  # red reference line

    # display the counters on the frame
    cvzone.putTextRect(frame, f'Cars Up: {len(upcar_counter)}', (20, 30), 1, 1, (0, 255, 0))
    cvzone.putTextRect(frame, f'Cars Down: {len(downcar_counter)}', (20, 60), 1, 1, (0, 0, 255))
    cvzone.putTextRect(frame, f'Buses Up: {len(upbus_counter)}', (20, 90), 1, 1, (0, 255, 0))
    cvzone.putTextRect(frame, f'Buses Down: {len(downbus_counter)}', (20, 120), 1, 1, (0, 0, 255))
    cvzone.putTextRect(frame, f'Trucks Up: {len(uptruck_counter)}', (20, 150), 1, 1, (0, 255, 0))
    cvzone.putTextRect(frame, f'Trucks Down: {len(downtruck_counter)}', (20, 180), 1, 1, (0, 0, 255))

    # calculate and display FPS
    end_time = time.time()
    elapsed_time = end_time - start_time
    fps = frame_count / elapsed_time

    cv2.putText(frame, f'FPS: {fps:.2f}', (800, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

    # display the updated frame
    cv2.imshow('window', frame)
```

```
    # display the updated frame
    cv2.imshow('window', frame)

    if cv2.waitKey(1) & 0xFF == 27: # press ESC to exit
        break

# release resources and close windows
capture.release()
cv2.destroyAllWindows()
```

Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

tracker.py

```python
import math


class Tracker:
    def __init__(self):
        self.center_points = {} # to store the center positions of the objects
        self.id_count = 0 # keep the count of the IDs - each time a new object id detected, count will increase by one

    def update(self, object_rectangles):
        objects_bbs_ids = [] # object bounding boxes and ids

        # get center point of new object
        for rectangle in object_rectangles:
            x, y, w, h = rectangle
            centre_x = (x + x + w) // 2
            centre_y = (y + y + h) // 2

            # find out if that object was detected already
            same_object_detected = False
            for id, pt in self.center_points.items():
                distance = math.hypot(centre_x - pt[0], centre_y - pt[1])

                if distance < 70:
                    self.center_points[id] = (centre_x, centre_y)
                    objects_bbs_ids.append([x, y, w, h, id])
                    same_object_detected = True
                    break

            # if a new object is detected - assign id to it
            if same_object_detected is False:
                self.center_points[self.id_count] = (centre_x, centre_y)
                objects_bbs_ids.append([x, y, w, h, self.id_count])
                self.id_count += 1

        # remove ids not being used anymore
        new_center_points = {}
        for obj_bb_id in objects_bbs_ids:
            _, _, _, _, object_id = obj_bb_id
            center = self.center_points[object_id]
            new_center_points[object_id] = center

        # update the dictionary
        self.center_points = new_center_points.copy()
        return objects_bbs_ids
```

Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

Code Structure and Components:

- **YOLO Model Loading**: The YOLOv8 pre-trained model is loaded for object detection, enabling accurate and fast detection of vehicles in each frame.
- **Mouse Position Tracking**: A utility to track the mouse position over the OpenCV window for testing and debugging.
- **Vehicle Tracking**: Custom tracking logic with unique IDs for each detected vehicle using a `Tracker` class, which updates vehicle positions across frames.
- **Direction Thresholds**: Two horizontal lines (green and red) serve as boundaries to track whether vehicles are moving up or down.
- **FPS Calculation**: The FPS calculation displays the frame processing speed on each frame to assess system performance in real-time.

Video Description:

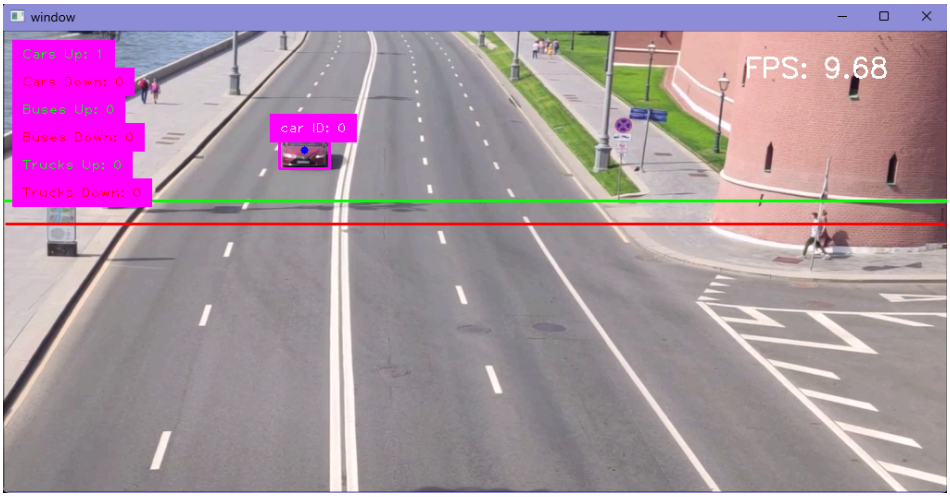
The video shows a highway segment where cars, buses, and trucks move in two lanes, either heading towards or away from the camera. The vehicles are detected and labeled in real-time, and their movements are tracked across two reference lines, allowing the system to determine if they are moving up or down.

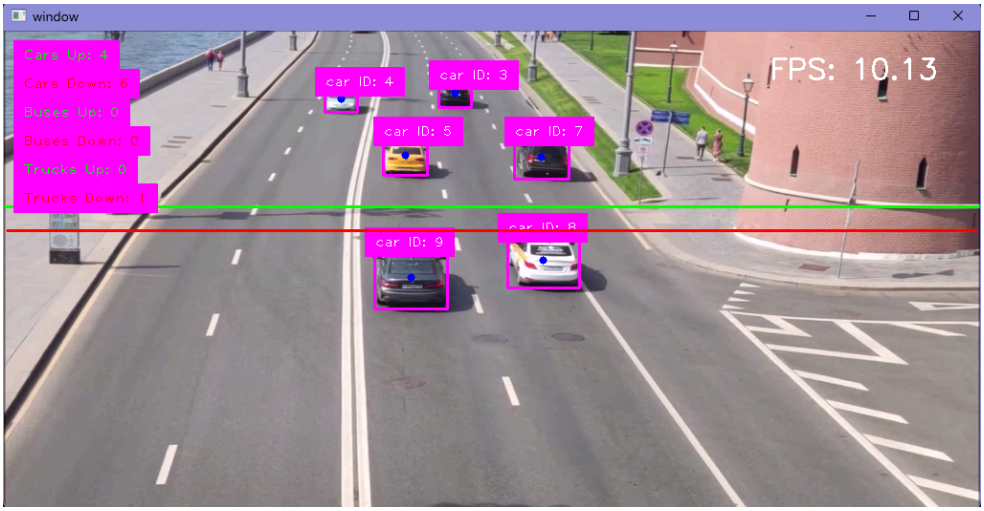
Department of Computer Science & Engineering (Data Science)

Image Processing and Computer Vision I (DJ22DSL503)

Input Video (Screenshots):

Output Video (Screenshots):





Frames Per Second (FPS) Calculation:

The FPS (Frames Per Second) is calculated to measure the system's performance and is displayed on each frame. This was achieved by counting frames and measuring the time elapsed since the beginning of the video.

```
elapsed_time = end_time - start_time
fps = frame_count / elapsed_time
```

On average, the FPS was 10.

Performance Evaluation:

The system performs well in detecting and distinguishing between vehicle types in real-time. With an FPS of around 10, it's sufficient for practical applications where ultra-high frame rates aren't required. However, performance may vary based on video resolution and hardware specifications.

Strengths:

- **Real-time Detection and Tracking**: The system can detect and track multiple vehicle types in real-time.
- **Accurate Classification:** YOLOv8 provides robust vehicle classification, accurately identifying cars, buses, and trucks.
- **Direction Counting:** The implementation of direction tracking and counting adds value for traffic monitoring purposes.

Limitations:

- **FPS Limitation**: With an FPS of around 10, the system might struggle with high-speed or dense traffic scenarios where higher frame rates are required.
- **Model Limitations**: YOLOv8's pretrained model might not perform optimally in all weather or lighting conditions, potentially impacting detection accuracy.
- **False Positives in Edge Cases**: Rapidly changing object positions (e.g., overlapping vehicles) might cause inaccuracies in direction tracking.

Conclusion:

This project successfully demonstrates the capability to monitor highway traffic in real-time, tracking and counting vehicles moving in different directions. The system could be valuable for traffic analytics, congestion monitoring, and smart city applications. Future improvements might focus on optimizing FPS, improving accuracy under challenging conditions, and extending the system to handle more complex traffic patterns.